

Prova 3 – Turma B
Estruturas de Dados Elementares e Árvores

Disciplina: PCC104 – Projeto e Análise de Algoritmos | DECOM – UFOP

25 de junho de 2026

Instruções. Esta prova contém 7 questões discursivas. Você deverá entregar o código das implementações da lista de exercícios impresso juntamente com a prova. Algumas das questões farão referência ao seu próprio código. A não entrega do código implica em nota 0 nestas questões.

1. (**Arrays.**) Escreva o pseudocódigo de um algoritmo que recebe um array $A[1..n]$ de inteiros **não necessariamente ordenado** e um valor x , e devolve o índice de x em A (ou NIL caso x não esteja presente). Apresente a análise de complexidade deste algoritmo no melhor e no pior caso.

2. (**Pilha com mínimo em $O(1)$.**) Considere sua implementação de MINSTACK. Execute a sequência de operações abaixo sobre uma instância inicialmente vazia, mostrando após cada chamada o estado das duas pilhas e o valor devolvido (quando houver):

PUSH(6), PUSH(9), PUSH(2), PUSH(4), MIN(), POP(), MIN(), POP(), MIN()

Em seguida, enuncie o invariante que sua estrutura auxiliar mantém e explique por que MIN() é $O(1)$ no pior caso.

3. (**Filas.**) Escreva o pseudocódigo de ENQUEUE(Q, x) e DEQUEUE(Q) para uma fila implementada com **duas pilhas** S_1 e S_2 (apenas operações PUSH, POP e teste de vazio). Indique o custo de pior caso e o custo amortizado de cada operação.

4. (**Listas duplamente encadeadas.**) Seja L uma lista duplamente encadeada com sentinela $L.nil$ no estilo CLRS, contendo n elementos. Justifique a complexidade de tempo de LIST-INSERT (inserção no início), LIST-DELETE (dado ponteiro x para o nó) e LIST-SEARCH (busca por chave k). Em seguida, explique por que LIST-DELETE é $O(n)$ em uma lista *simplesmente* encadeada sem acesso ao predecessor, e descreva o truque que permite $O(1)$ nesse caso, apontando quando ele falha.

5. (**Tabelas hash.**) O pseudocódigo abaixo apresenta o algoritmo de inserção em uma tabela hash com encadeamento separado, onde cada posição aponta para uma lista encadeada e a função hash é dada por $h(k) = k \bmod m$.

```
1: procedure HASH-INSERT( $T, k$ )
2:    $j \leftarrow k \bmod m$ 
3:   cria novo nó  $z$  com  $z.key = k$ 
4:    $z.next \leftarrow T[j]$ 
5:    $T[j] \leftarrow z$ 
6: end procedure
```

Execute HASH-INSERT para cada chave da sequência

5, 14, 19, 3, 26, 11, 33

em uma tabela com $m = 7$, mostrando o estado completo de T após cada inserção.

6. (**Altura de uma árvore binária de busca.**)

- (a) Escreva o pseudocódigo de TREE-HEIGHT(x), que recebe a raiz x de uma ABB e devolve sua altura (árvore vazia = -1, nó folha = 0). Apresente a versão recursiva, identificando caso base e passo recursivo.
- (b) Formule a recorrência $T(n)$ no pior caso, resolva-a e expresse o resultado em notação Θ .
- (c) Uma ABB é construída inserindo as chaves 8, 3, 12, 1, 5, 10, 14 nessa ordem. Desenhe a árvore e aplique TREE-HEIGHT à raiz, mostrando os valores devolvidos em cada chamada recursiva.

7. (**Árvores rubro-negras.**) Quando devemos utilizar árvores rubro-negras?